Frontend – Assignment 1

Complete Solutions

Q1. HTTP Request–Response Cycle

The HTTP Request–Response Cycle is the process through which a client such as a web browser communicates with a web server.

Diagram:
Browser/Client
↓
1. DNS Lookup
↓
2. Establish Connection (TCP/TLS)
↓
3. HTTP Request
↓
Web Server
↓
4. Server Processes Request
↓
5. HTTP Response
↓
Browser receives HTML/CSS/JS/data
↓
6. Browser renders the webpage

Major steps:
1. The user enters a URL or clicks a link.
2. The browser resolves the domain name to an IP address using DNS.
3. A connection is established with the server, usually using TCP and, for HTTPS, TLS.
4. The browser sends an HTTP request containing a method such as GET or POST, the URL/path, headers, and sometimes a body.
5. The web server receives and processes the request. It may access application logic or a database.
6. The server sends an HTTP response containing a status code, headers, and response data such as HTML, JSON, CSS, or JavaScript.
7. The browser processes the response and renders the webpage.

Q2. Introduction to Web Technologies

1. HTML (HyperText Markup Language): Defines the structure and content of a webpage using elements such as headings, paragraphs, links, images, forms, and tables.

2. CSS (Cascading Style Sheets): Controls presentation and layout, including colors, fonts, spacing, borders, positioning, and responsive design.

3. JavaScript: Adds programming logic and interactivity to webpages, such as event handling, validation, dynamic content, and API communication.

4. Bootstrap: A frontend CSS framework that provides reusable components, responsive grid/layout utilities, and ready-made UI styles.

5. jQuery: A JavaScript library that simplified DOM manipulation, event handling, animations, and AJAX operations, especially in older web applications.

6. React.js: A JavaScript library for building component-based user interfaces. It helps developers create reusable UI components and efficiently update dynamic interfaces.

Q3. Client-Side vs. Server-Side Technologies

Client-side technologies run mainly in the user's browser. They are responsible for the user interface, presentation, and interactions.

Examples: HTML, CSS, JavaScript, React.js.

Server-side technologies run on the web server. They handle business logic, authentication, database operations, and generating or serving data.

Examples: Node.js, Java, Python, PHP, and .NET.

Key differences:
Client-side → runs in browser, focuses on UI/interactivity, code is generally delivered to the client.
Server-side → runs on server, handles business logic/data/security, and sends a response to the client.

Q4. Block-Level Elements in HTML

Block-level elements normally start on a new line and occupy the available width of their containing block. They are commonly used to structure the major sections of a webpage.

Characteristics:
• Start on a new line in normal HTML flow.
• Usually take the full available width.
• Can contain other elements depending on HTML rules.
• Useful for creating page structure and sections.

Examples: , <p>, <h1> to <h6>, <section>, <article>, and <header>.

Q5. Semantic HTML Elements

Semantic elements clearly describe the meaning and purpose of their content. They make HTML easier for developers, browsers, and assistive technologies to understand.

Importance:
• Improves code readability and maintainability.
• Helps accessibility tools understand page structure.
• Provides meaningful document structure.
• Can improve search-engine understanding of page content.

Examples: <header>, <nav>, <main>, <section>, <article>, <aside>, and <footer>.

Q6. Creating a Table Using HTML

HTML code:

```
&lt;table border="1"&gt;
  &lt;thead&gt;
    &lt;tr&gt;
      &lt;th&gt;Sr. No.&lt;/th&gt;
      &lt;th&gt;Name&lt;/th&gt;
      &lt;th&gt;Designation&lt;/th&gt;
      &lt;th&gt;Department&lt;/th&gt;
    &lt;/tr&gt;
  &lt;/thead&gt;
```

```
  &lt;tbody&gt;
    &lt;tr&gt;
      &lt;td&gt;1&lt;/td&gt;
      &lt;td&gt;John&lt;/td&gt;
      &lt;td&gt;Manager&lt;/td&gt;
      &lt;td&gt;Sales&lt;/td&gt;
    &lt;/tr&gt;
    &lt;tr&gt;
      &lt;td&gt;2&lt;/td&gt;
      &lt;td&gt;Smith&lt;/td&gt;
      &lt;td&gt;Sr. Manager&lt;/td&gt;
      &lt;td&gt;Marketing&lt;/td&gt;
    &lt;/tr&gt;
    &lt;tr&gt;
      &lt;td&gt;3&lt;/td&gt;
      &lt;td&gt;Jane&lt;/td&gt;
      &lt;td&gt;Manager&lt;/td&gt;
      &lt;td&gt;HR&lt;/td&gt;
    &lt;/tr&gt;
  &lt;/tbody&gt;
&lt;/table&gt;
```

Q7. Creating a Nested List Using HTML

```
HTML code:

&lt;ul&gt;
  &lt;li&gt;River
    &lt;ul&gt;
      &lt;li&gt;Ganga&lt;/li&gt;
      &lt;li&gt;Yamuna&lt;/li&gt;
      &lt;li&gt;Brahmaputra&lt;/li&gt;
      &lt;li&gt;Narmada&lt;/li&gt;
    &lt;/ul&gt;
  &lt;/li&gt;
&lt;/ul&gt;
```

Q8. Types of CSS

There are three common ways to apply CSS:

1. Inline CSS
CSS is written directly inside an HTML element using the style attribute.
Example:
Hello

2. Internal CSS
CSS is written inside a <style> element in the HTML document, usually inside <head>.
Example:
<style>
p { color: blue; }
</style>

3. External CSS
CSS is written in a separate .css file and connected using <link>.
HTML:
<link rel="stylesheet" href="style.css">

style.css:
p {
color: blue;
}

External CSS is generally preferred for larger projects because it promotes reuse and separation of concerns.

Q9. Practical Understanding

HTML, CSS, and JavaScript have different but complementary roles:

HTML → Structure: Creates headings, paragraphs, images, forms, tables, links, and other webpage content.

CSS → Presentation: Controls colors, typography, spacing, layout, responsiveness, and visual appearance.

JavaScript → Behavior: Adds interaction and logic, such as button actions, form validation, dynamic updates, calculations, and fetching data.

In simple terms:
HTML = skeleton
CSS = appearance
JavaScript = behavior/interactivity

Q10. Short Answer

a. Role of Bootstrap:
Bootstrap is a frontend framework that provides responsive layout utilities, a grid system, and reusable UI components. It helps developers build consistent and mobile-friendly interfaces faster.

b. jQuery:
jQuery is a JavaScript library designed to simplify common tasks such as DOM manipulation, event handling, animations, and AJAX. It became widely used because it reduced the amount of code required and helped handle browser differences in older browsers.

c. React.js:
React.js is a JavaScript library for building user interfaces using reusable components. It helps solve the problem of efficiently developing and updating complex, dynamic frontend interfaces by organizing the UI into components and efficiently managing UI updates.